Laporan Tugas Besar

IF3170 Inteligensi Artifisial

Mejiro Express



Disusun Oleh :

Kapal Karam Oddysseus

Raysha Erviandika Putra	13524050
Zeki Amani	13524082
Muhammad Daffa Arrizki Yanma	13524133

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Deskripsi Persoalan

Mejiro Express adalah perusahaan yang bergerak dalam kegiatan pengiriman paket. Permasalahan pada tugas ini adalah menentukan konfigurasi pengisian paket ke dalam container truk agar total nilai paket yang diangkut menjadi maksimal.

Setiap paket memiliki atribut:

1. Identitas paket.
2. Dimensi panjang, lebar, dan tinggi.
3. Posisi dalam koordinat tiga dimensi.
4. Enam kemungkinan orientasi.
5. Nilai paket.
6. Berat paket.
7. Status barang pecah-belah.
8. *Estimated Time of Arrival* (ETA).
9. Status keberadaan paket di dalam atau di luar truk.

Suatu konfigurasi dinyatakan valid apabila seluruh paket memenuhi *constraint* sebagai berikut:

1. Seluruh bagian paket berada di dalam batas container.
2. Paket tidak saling bertumpang tindih.
3. Paket tidak melayang dan harus memiliki penopang.
4. Paket pecah-belah tidak boleh menopang paket lain.
5. Total berat paket tidak melebihi kapasitas maksimum truk.
6. Paket harus berada sepenuhnya di dalam atau sepenuhnya di luar *container*.

Fungsi objektif yang digunakan adalah jumlah nilai seluruh paket di dalam container. Karena banyaknya kemungkinan posisi, orientasi, dan kombinasi paket, pencarian menyeluruh menjadi tidak efisien. Oleh karena itu, digunakan algoritma local search, yaitu Hill Climbing, Simulated Annealing, dan Genetic Algorithm, untuk memperoleh konfigurasi dengan nilai terbaik. Hasil pencarian ditampilkan melalui visualisasi state awal, perubahan pada setiap iterasi, state akhir, dan perkembangan nilai fungsi objektif

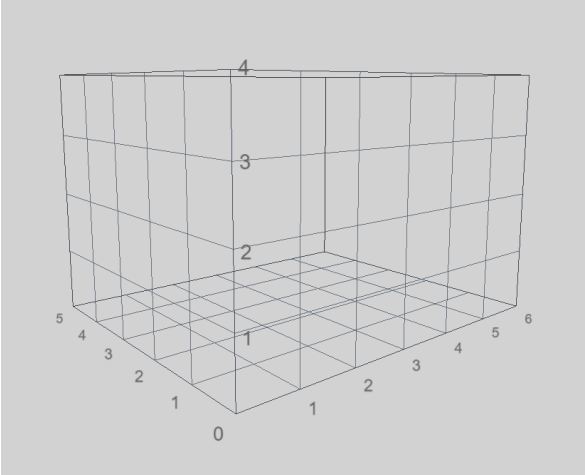
Pembahasan

1. Representasi Persoalan

1.1. Representasi State

State merepresentasikan kombinasi dari konfigurasi paket dalam kontainer truk, setiap state disimpan dalam struktur file JSON yang terdiri atas:

- Truck
  - Dimensi {Lebar, Panjang, Tinggi}
  - Kapasitas (Berat maksimum)
- Package
  - ID Paket
  - Dimensi awal paket
  - Posisi (x, y, z)
  - Orientasi paket dengan nilai 0–5
  - Nilai dan berat paket.
  - Kategori *fragile* atau tidak
  - ETA.
  - inTruck untuk posisi paket



Gambar 1.1 Representasi Ruang Truk dalam Grid

Posisi (0,0,0) berada pada salah satu sudut container truk, sumbu x merepresentasikan lebar (w), y merepresentasikan tinggi (h) dan z merepresentasikan panjang (l). Orientasi adalah kumpulan kombinasi dimensi paket yang digunakan untuk mimik rotasi. Terdapat enam kemungkinan kombinasi rotasi dari (w, l, h) yaitu:

- 0: (w, l, h)
- 1: (w, h, l)
- 2: (h, w, l)
- 3: (h, l, w)
- 4: (l, w, h)
- 5: (l, h, w)

Ketika rotasi dilakukan yang sebenarnya terjadi adalah mengubah dimensi panjang, tinggi, dan lebar pada paket yang sama, hal ini juga untuk memudahkan rendering.

Berikut adalah contoh 1 konfigurasi state:

```
{
  "truck": {
    "dimensions": { "w": 4, "l": 5, "h": 3 },
    "maxCapacity": 100
  },
  "packages": [
    {
      "id": "S01",
      "dimensions": { "w": 2, "l": 2, "h": 1 },
      "position": { "x": 0, "y": 0, "z": 0 },
      "orientation": 0,
      "value": 40,
      "weight": 20,
      "isFragile": false,
      "eta": 2,
      "inTruck": false
    }
  ]
}
```

Suatu state dapat dibangkitkan secara random untuk digunakan sebagai initial state. Pembangkitan ini diimplementasikan melalui fungsi create_random_individual yang menerima sebuah konfigurasi state dan

python random object untuk menghasilkan sebuah state random yang valid. Berikut implementasinya (beserta beberapa helper function),

```
def create_random_individual(board: dict, rng: Random) -> dict:
    individual = empty_state(board)
    package_indices = list(range(len(individual["packages"])))
    rng.shuffle(package_indices)

    for package_index in package_indices:
        if rng.random() < 0.50:
            find_random_valid_placement(individual, package_index, rng)

    return individual
def find_random_valid_placement(state: dict, package_index: int, rng:
Random) -> bool:
    truck_dimensions = state["truck"]["dimensions"]
    orientations = list(range(6))
    rng.shuffle(orientations)

    for orientation in orientations:
        package = deepcopy(state["packages"][package_index])
        package["orientation"] = orientation
        dimensions = curr_dimension(package)
        max_x = truck_dimensions["w"] - dimensions["w"]
        max_y = truck_dimensions["h"] - dimensions["h"]
        max_z = truck_dimensions["l"] - dimensions["l"]

        if max_x < 0 or max_y < 0 or max_z < 0:
             continue

        positions = [
             {"x": x, "y": y, "z": z}
             for x in range(max_x + 1)
             for y in range(max_y + 1)
             for z in range(max_z + 1)
        ]
        rng.shuffle(positions)

        for position in positions:
             candidate = deepcopy(state)
             if place_at(candidate, package_index, orientation,
position):
             state.clear()
             state.update(candidate)
             return True

       return False
def empty_state(board: dict) -> dict:
    state = deepcopy(board)
    for package in state["packages"]:
        package["inTruck"] = False
```

```
    return state


def place_at(
    state: dict,
    package_index: int,
    orientation: int,
    position: dict[str, int],
) -> bool:
    package = state["packages"][package_index]
    package["orientation"] = orientation
    package["position"] = position
    package["inTruck"] = True
       return isvalid(state)
```

1.2. Pembakitan *Successor*

Successor diperoleh dengan melakukan satu perubahan terhadap state saat ini. Perubahan tersebut dilakukan menggunakan salah satu operasi berikut:

- Swap, yaitu menukar posisi dua paket di dalam truk atau menukar paket di dalam dengan paket di luar truk.
- Move, yaitu memindahkan sebuah paket ke koordinat (x, y, z) yang berbeda. Operasi ini juga dapat digunakan untuk memindahkan paket dari luar ke dalam truk.
- Rotate, yaitu mengubah orientasi sebuah paket ke salah satu orientasi lain.

Swap antara dua paket yang sama-sama berada di luar truk tidak dilakukan karena tidak bermanfaat.

Setiap kandidat hasil perubahan diperiksa terhadap constraint berikut:

- Total berat paket tidak melebihi kapasitas truk.
- Seluruh bagian paket berada di dalam batas dimensi truk.
- Tidak terdapat dua paket yang saling *overlap*.
- Paket yang tidak berada di dasar memiliki penyangga.
- Paket tidak diletakkan di atas paket fragile.

Kandidat yang melanggar salah satu constraint tidak dimasukkan ke dalam kumpulan successor. Dengan demikian, hanya state valid yang akan dinilai menggunakan fungsi heuristic.

Seluruh variasi Hill Climbing menggunakan jenis operasi neighbor yang sama. Perbedaannya terletak pada cara memilih successor dari kumpulan neighbor tersebut.

Pada kode program, pembangkitan successor dilakukan menggunakan 2 fungsi yang berbeda yaitu:

1.2.1. generate_neighbors(state)

Digunakan oleh Steepest Ascent, Sideways Move, dan Random Restart Hill Climbing untuk membangkitkan seluruh neighbor yang valid melalui tiga tahap, yaitu pertukaran dua paket, pemindahan posisi paket, dan perubahan orientasi paket.

```
def generate_neighbors(state: dict) -> list[dict]:
    neighbors = []
    packages = state["packages"]
    truck_dim = state["truck"]["dimensions"]

    for i in range(len(packages)):
        for j in range(i + 1, len(packages)):
             if not packages[i].get("inTruck", False) and not packages[j].get("inTruck", False):
             continue
             candidate = deepcopy(state)
             a, b = candidate["packages"][i], candidate["packages"][j]
             a["position"], b["position"] = b["position"], a["position"]
             a["inTruck"], b["inTruck"] = b.get("inTruck", False), a.get("inTruck", False)
             if candidate != state and isvalid(candidate):
             neighbors.append(candidate)

    for i, package in enumerate(packages):
        dim = curr_dimension(package)
        for x in range(truck_dim["w"] - dim["w"] + 1):
             for y in range(truck_dim["h"] - dim["h"] + 1):
             for z in range(truck_dim["l"] - dim["l"] + 1):
                    position = {"x": x, "y": y, "z": z}
                    if package.get("inTruck", False) and package["position"] == position:
                           continue
                    candidate = deepcopy(state)
                    candidate["packages"][i]["position"] = position
                    candidate["packages"][i]["inTruck"] = True
                    if isvalid(candidate):
                        neighbors.append(candidate)

    for i, package in enumerate(packages):
        for orientation in range(6):
             if orientation == package.get("orientation", 0):
             continue
             candidate = deepcopy(state)
            candidate["packages"][i]["orientation"] = orientation
             if isvalid(candidate):
             neighbors.append(candidate)

    return neighbors
```

1.2.2. gen_random_suc(curr)

Digunakan oleh Stochastic Hill Climbing dan Simulated Annealing untuk membangkitkan satu random successor yang valid dengan melakukan aksi satu aksi secara acak kepada

suatu paket acak hingga diperoleh state kandidat yang memenuhi valid.

```
def gen_random_suc(curr):
    while(True):
        i = choice([k for k, _ in enumerate(curr["packages"])])
        package= curr["packages"][i]
        truck_dim = curr["truck"]["dimensions"]

        action = choice(["swap", "move", "rotate"])

        orientation= choice([o for o in range(6) if o != package["orientation"]]) if
action=="rotate" else package["orientation"]
        rotated = deepcopy(package)
        rotated["orientation"] = orientation
        dim = curr_dimension(rotated)

        if(truck_dim["w"] - dim["w"] + 1<=0 or truck_dim["h"] - dim["h"] + 1<=0 or
truck_dim["l"] - dim["l"] + 1<=0):
             continue
        position = package["position"]
        if(action=="move"):
            x=randrange(truck_dim["w"] - dim["w"] + 1)
            y=randrange(truck_dim["h"] - dim["h"] + 1)
            z=randrange(truck_dim["l"] - dim["l"] + 1)
             position = {"x": x, "y": y, "z": z}

        candidate = deepcopy(curr)
        candidate["packages"][i]["orientation"] = orientation
        candidate["packages"][i]["position"] = position

        if(action=="swap"):
            if(len(curr["packages"]) < 2):
             continue
             j = choice([k for k, _ in enumerate(curr["packages"]) if k != i])
             a, b = candidate["packages"][i], candidate["packages"][j]
             for key in ["position", "orientation", "inTruck"]:
             a[key], b[key] = b[key], a[key]

        if(isvalid(candidate)):
             break
    return candidate
```

Dengan valid didefinisikan sebagai kondisi state yang memenuhi constraint yang mana mencangkup pemeriksaan kapasitas berat maksimum truk, batas dimensi kontainer (x, y, z), pengecekan penopang dan paket fragile. isvalid() diimplementasikan sebagai berikut (beserta beberapa helper function yang releavan),

```
def isvalid(state: dict) -> bool:
    truck = state["truck"]
    truck_dim = truck["dimensions"]

    inside = [
        package for package in state["packages"] if package.get("inTruck", True)
    ]

    total_weight = sum(package["weight"] for package in inside)

    if total_weight > truck["maxCapacity"]:
        return False

    for package in inside:
        pos = package["position"]
        dim = curr_dimension(package)

        if pos["x"] < 0 or pos["y"] < 0 or pos["z"] < 0:
             return False

        if pos["x"] + dim["w"] > truck_dim["w"]:
             return False

        if pos["y"] + dim["h"] > truck_dim["h"]:
             return False

        if pos["z"] + dim["l"] > truck_dim["l"]:
             return False

        if raycast_below_for_fragile_or_support(state, package):
             return False

    for i in range (len(inside)):
        for j in range(i + 1, len(inside)):
             if cek_overlap(inside[i], inside[j]):
             return False

    return True

def raycast_below_for_fragile_or_support(state: dict, package: dict) -> bool:
    pos = package["position"]
    dim = curr_dimension(package)

    if pos["y"] == 0:
        return False

    has_support = False

    for x in range(pos["x"], pos["x"] + dim["w"]):
        for z in range(pos["z"], pos["z"] + dim["l"]):
             for other in state["packages"]:
             if other["id"] == package["id"]:
                    continue
```

```
            if not other.get("inTruck", True):
                  continue

            other_pos = other["position"]
            other_dim = curr_dimension(other)

            directly_below = other_pos["y"] + other_dim["h"] == pos["y"]
            covers_point = (
                  other_pos["x"] <= x < other_pos["x"] + other_dim["w"]
                  and other_pos["z"] <= z < other_pos["z"] + other_dim["l"]
            )

            if directly_below and covers_point:
                  if other.get("isFragile", False):
                        return True

                  has_support = True
                  break

    return not has_support

def cek_overlap(a: dict, b: dict) -> bool:
    a_pos = a["position"]
    a_dim = curr_dimension(a)
    b_pos = b["position"]
    b_dim = curr_dimension(b)

    overlap_x = not (a_pos["x"] + a_dim["w"] <= b_pos["x"] or b_pos["x"] + b_dim["w"] <=
a_pos["x"])
    overlap_y = not (a_pos["y"] + a_dim["h"] <= b_pos["y"] or b_pos["y"] + b_dim["h"] <=
a_pos["y"])
    overlap_z = not (a_pos["z"] + a_dim["l"] <= b_pos["z"] or b_pos["z"] + b_dim["l"] <=
a_pos["z"])

    return overlap_x and overlap_y and overlap_z

def curr_dimension(package: dict) -> dict:
    w, h, l = package["dimensions"]["w"], package["dimensions"]["h"],
package["dimensions"]["l"]

    orientations = [
        {"w": w, "l": l, "h": h},
        {"w": w, "l": h, "h": l},
        {"w": h, "l": w, "h": l},
        {"w": h, "l": l, "h": w},
        {"w": l, "l": w, "h": h},
        {"w": l, "l": h, "h": w},
    ]

    orientation = package.get("orientation", 0)

    if orientation < 0 or orientation >= len(orientations):
        raise ValueError(f"idk why I need this")
```

```
return orientations[orientation]
```

1.3. Scoring dan Heuristic

Fungsi heuristic digunakan untuk mengukur kualitas sebuah state. Semakin besar nilai heuristic, semakin baik konfigurasi paket pada state tersebut.
Terdapat dua pilihan objective function yang dapat digunakan.

- Objective function utama menghitung *value* paket yang berada di dalam truk.

$$h(s) = \sum value(p)$$

- Sebagai objective function alternatif, ETA digunakan untuk memberikan prioritas lebih besar kepada paket yang harus dikirim lebih dahulu. Paket dengan ETA lebih kecil akan memperoleh tambahan nilai yang lebih besar.

$$h(s) = \sum \left(value(p) + \frac{value(p)}{\max(ETA(p), 1)} \right)$$

Setelah successor valid dibangkitkan, setiap successor dihitung menggunakan objective function yang dipilih. Hasil score tersebut kemudian digunakan oleh algoritma untuk menentukan state berikutnya.

Implementasi fungsi objektif pada program dilakukan dengan fungsi scoring() sebagai berikut,

```
def scoring(state: dict) -> float:
    packages = [package for package in state["packages"] if package.get("inTruck", True)]

    if state.get("scoringMode") == "value_eta":
        return sum(
             package["value"] + package["value"] / max(package.get("eta", 1), 1)
             for package in packages
        )

    return sum(package["value"] for package in packages)
```

2. Test Case

2.1. local_optimum

Konfigurasi berada di testcase/local_optimum.json yang berisi konfigurasi dimana ukuran dan weight package sengaja dibuat agar memiliki beberapa local optimum dengan global optimum bernilai 128.

2.2. asalklik

Konfigurasi berada di testcase/asalklik.json yang berisi konfigurasi dimana jumlah package dibuat sangat banyak. Testcase ini digunakan untuk membandingkan kecepatan eksekusi dari masing masing algoritma.

2.3. shoulder

Konfigurasi berada di test_cases/shoulder.json yang berisi konfigurasi di mana dimensi truk dan ukuran package sengaja dibuat agar terbentuk shoulder pada objective function (pada seed=2). Testcase ini digunakan untuk menunjukkan keberhasilan algoritma Hill Climbing with Sideways Move dalam melewati shoulder yang membuat Steepest Ascent Hill Climbing terjebak.

3. Steepest Ascent Hill Climbing

Steepest Ascent Hill Climbing memilih state terbaik dari seluruh successor valid yang berhasil dibangkitkan. Pada setiap iterasi, semua neighbor dinilai menggunakan fungsi heuristic. Neighbor dengan skor tertinggi dipilih sebagai state berikutnya apabila skornya lebih besar daripada state saat ini. Pencarian berhenti ketika tidak terdapat successor yang lebih baik atau ketika jumlah iterasi maksimum tercapai.

Implementasi dari algoritma ini adalah sebagai berikut.

```
def solve_steepest(board: dict, max_iter: int = 100) -> dict:
    curr = create_random_individual(board,_inst)
    init = deepcopy(curr)
    history = []

    for iteration in range(0, max_iter):
        neighbors = generate_neighbors(curr)
        if not neighbors:
             break

        best = max(neighbors, key=scoring)

        if scoring(best) <= scoring(curr):
             break

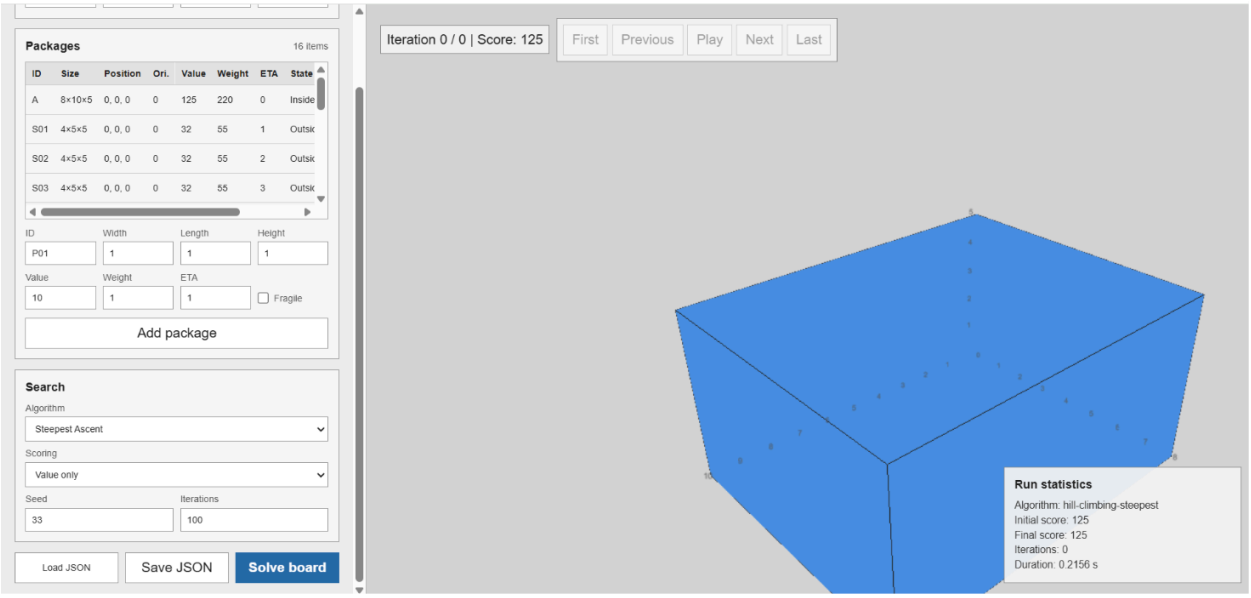
        curr = best

        history.append({
             "iteration" : iteration + 1,
             "score" : scoring(curr),
             "state" : deepcopy(curr)
        })

    return {
        "algorithm" : "hill-climbing-steepest",
        "implemented" : True,
        "init" : init,
        "steps" : history,
        "final" : curr,
        "initScore" : scoring(init),
        "finalScore" : scoring(curr),
        "iterations" : len(history)
                 }
```

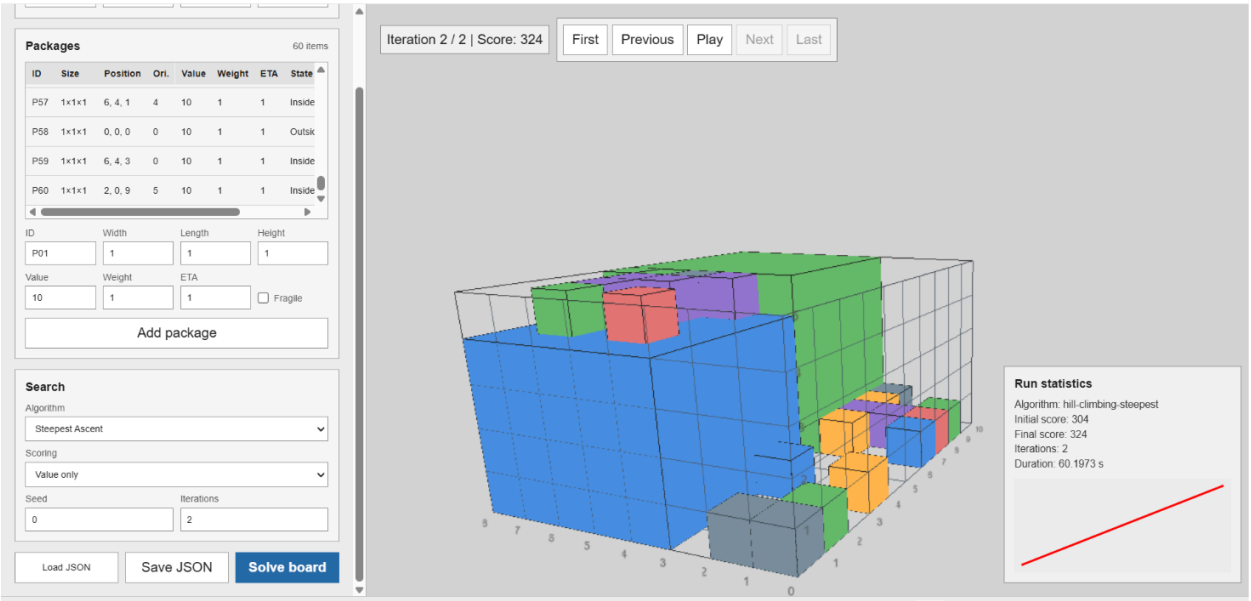
Hasil Pengujian & Analisis:

1. **local_optima**



Berdasarkan hasil pengujian pada test case local_optimum, algoritma steepest Hill Climbing pada seed 33, atau bahkan dengan seed lainnya juga, mengalami stuck pada local optimum bernilai 125 padahal global optimum nya adalah 128.

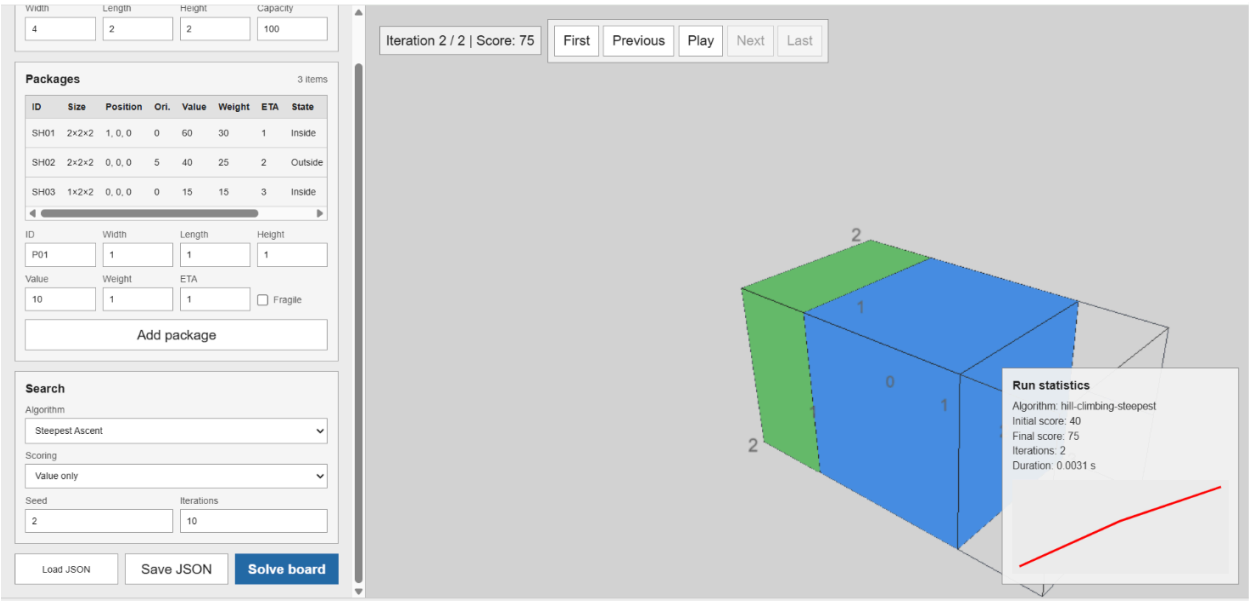
2. asalklik



Berdasarkan hasil pengujian pada test case asalklik dengan seed 0, algoritma steepest Hill Climbing memiliki waktu eksekusi untuk iterasi yang dibatasi sampai 2

iterasi saja sudah membutuhkan waktu yang sangat lama. Hal ini kemungkinan besar disebabkan steepest ascent hill climbing yang perlu membangkitkan semua sucsessor menyebabkan waktu eksekusi yang sangat lama.

3. shoulder



Berdasarkan hasil pengujian pada test case shoulder dengan seed 2, algoritma Steepest Ascent Hill Climbing mengalami stuck pada local optimum, yaitu pada score 75.

4. Sideway Hill Climbing

Sideway Hill Climbing melakukan hal yang sama dengan Steepest Hill Climbing hanya saja program tidak akan berhenti walaupun ketika mencapai shoulder alias program baru berhenti ketika neighbor.value<current.value dengan neighbor adalah successor dengan value/score tertinggi.

Algoritma ini diimplementasikan sebagai fungsi berikut,

```
def solve_sideways(board: dict, max_iter: int = 100, max_sideways: int = 20) -> dict:
    curr = create_random_individual(board,_inst)
    init = deepcopy(curr)
    history = []

    for iteration in range(0, max_iter):
        neighbors = generate_neighbors(curr)
        if not neighbors:
             break

        best = max(neighbors, key=scoring)

        if scoring(best) < scoring(curr):
             break

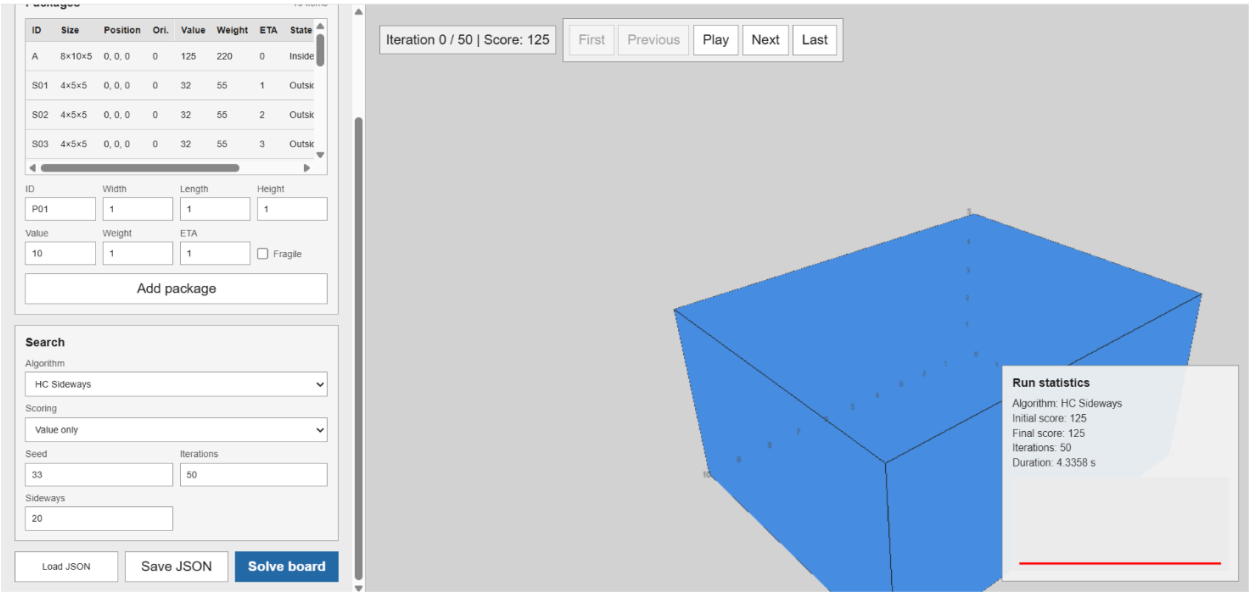
        curr = best

        history.append({
             "iteration" : iteration + 1,
             "score" : scoring(curr),
             "state" : deepcopy(curr)
        })

    return {
        "init" : init,
        "steps" : history,
        "final" : curr,
        "initScore" : scoring(init),
        "finalScore" : scoring(curr),
        "iterations" : len(history)
      }
```

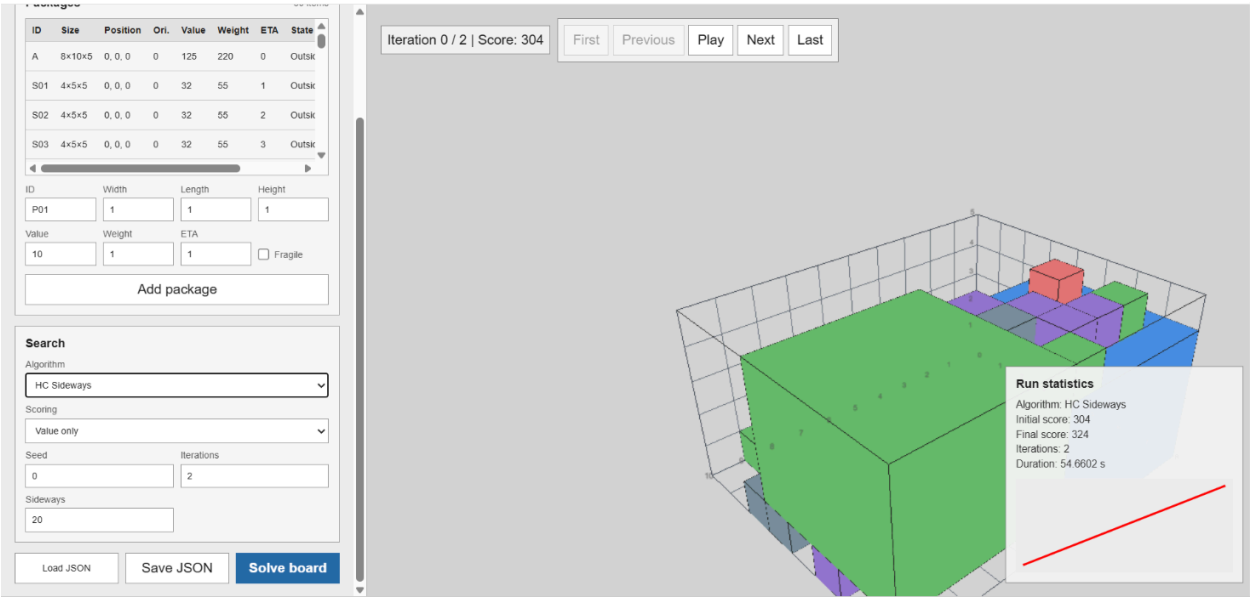
Hasil Pengujian & analisis:

1. **local_optima**



Berdasarkan hasil pengujian test case local_optimum, algoritma Sideway Hill Climbing walaupun sudah melakukan sideway search pun hasilnya tetap sama dengan steepest ascent hill climbing yaitu mengalami stuck pada local optimum dengan score 125 karena tidak melakukan random walk.

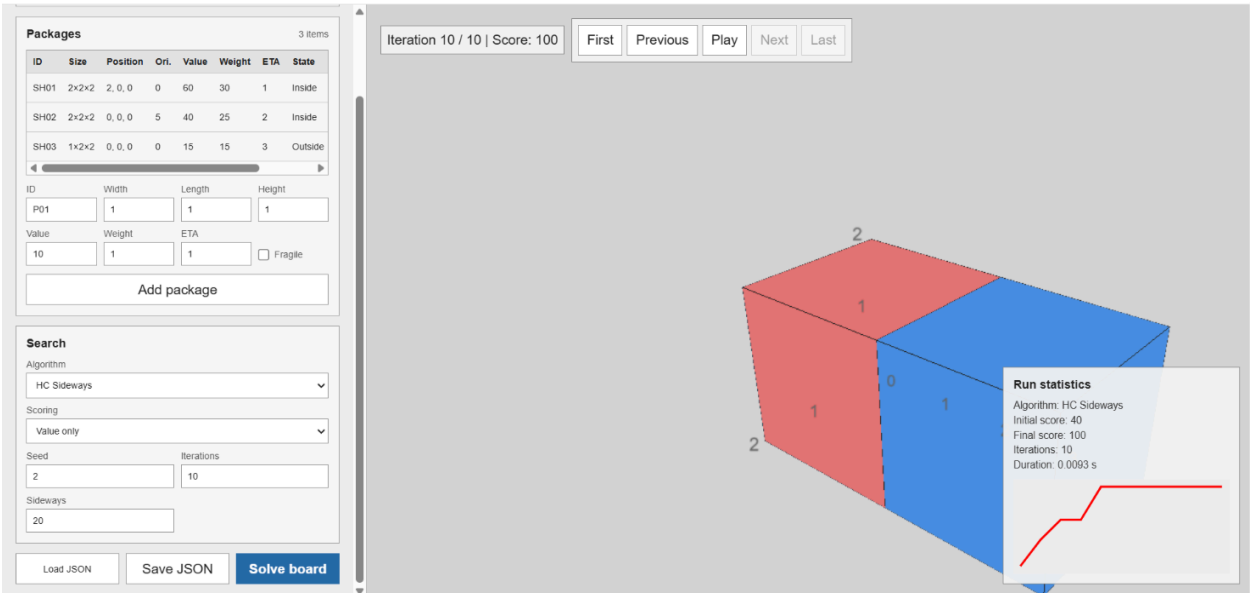
2. asalklik



Berdasarkan hasil pengujian pada test case asalklik dengan algoritma sideway Hill Climbing pada seed 0 terlihat bahwa waktu eksekusi untuk iterasi yang dibatasi

sampai 2 iterasi saja sudah membutuhkan waktu yang sangat lama. Penyebab lama eksekusi ini juga sama dengan pada kasus steepest ascent hill climbing

3. shoulder



Berdasarkan hasil pengujian pada test case shoulder dengan test case 2, algoritma sideway Hill Climbing berhasil escape dari local optimum karena melewati shoulder dan mencapai global optimum.

5. Random Restart Hill Climbing

Random Restart Hill Climbing menjalankan pencarian Steepest Ascent Hill Climbing secara berulang kali hingga sejumlah restart yang dipilih (max_restart) dengan state awal acak baru pada setiap restart.

Algoritma ini diimplementasikan dengan fungsi berikut,

```
def solve_random_restart(board: dict, max_iter: int = 100, max_restart: int = 10) -> dict:
    started_at = perf_counter()
    curr = None
    init = None
    history = []
    restart_stats = []
    total_iterations = 0

    for restart_number in range(1, max_restart + 1):
        restart_started_at = perf_counter()
        curr = create_random_individual(board,_inst)
        if init is None:
             init = deepcopy(curr)

        history.append({
             "iteration": len(history) + 1,
             "restart": restart_number,
             "restartIteration": 0,
             "restartStarted": True,
             "score": scoring(curr),
             "state": deepcopy(curr),
        })

        iterations_this_restart = 0
        for restart_iteration in range(1, max_iter + 1):
             iterations_this_restart = restart_iteration
             total_iterations += 1
             neighbors = generate_neighbors(curr)
             if not neighbors:
             break

             best = max(neighbors, key=scoring)

             if scoring(best) <= scoring(curr):
             break

             curr = best

             history.append({
             "iteration": len(history) + 1,
             "restart": restart_number,
             "restartIteration": restart_iteration,
             "restartStarted": False,
             "score": scoring(curr),
             "state": deepcopy(curr),
             })

        restart_stats.append({
             "restart": restart_number,
             "iterations": iterations_this_restart,
             "score": scoring(curr),
             "duration": perf_counter() - restart_started_at,
```

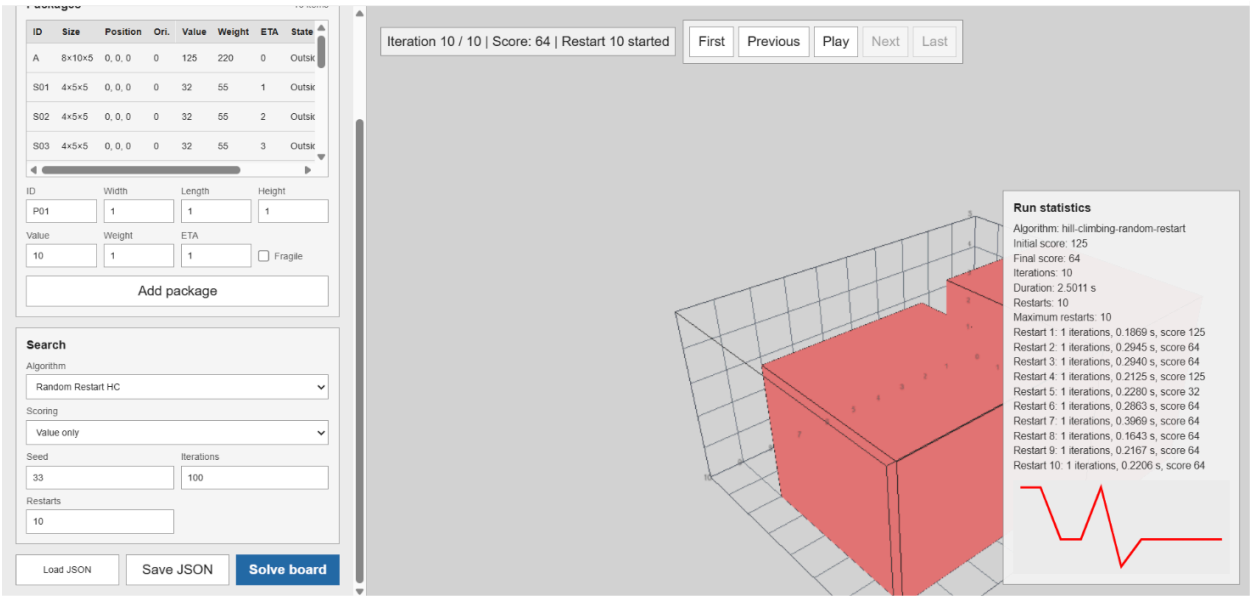
```
        })

    if init is None:
        curr = create_random_individual(board,_inst)
        init = deepcopy(curr)

    return {
        "algorithm" : "hill-climbing-random-restart",
        "implemented" : True,
        "init" : init,
        "steps" : history,
        "final" : curr,
        "initScore" : scoring(init),
        "finalScore" : scoring(curr),
        "iterations" : total_iterations,
        "restarts": len(restart_stats),
        "maxRestart": max_restart,
        "iterationsPerRestart": [item["iterations"] for item in restart_stats],
        "restartStats": restart_stats,
        "duration": perf_counter() - started_at,
    }
```

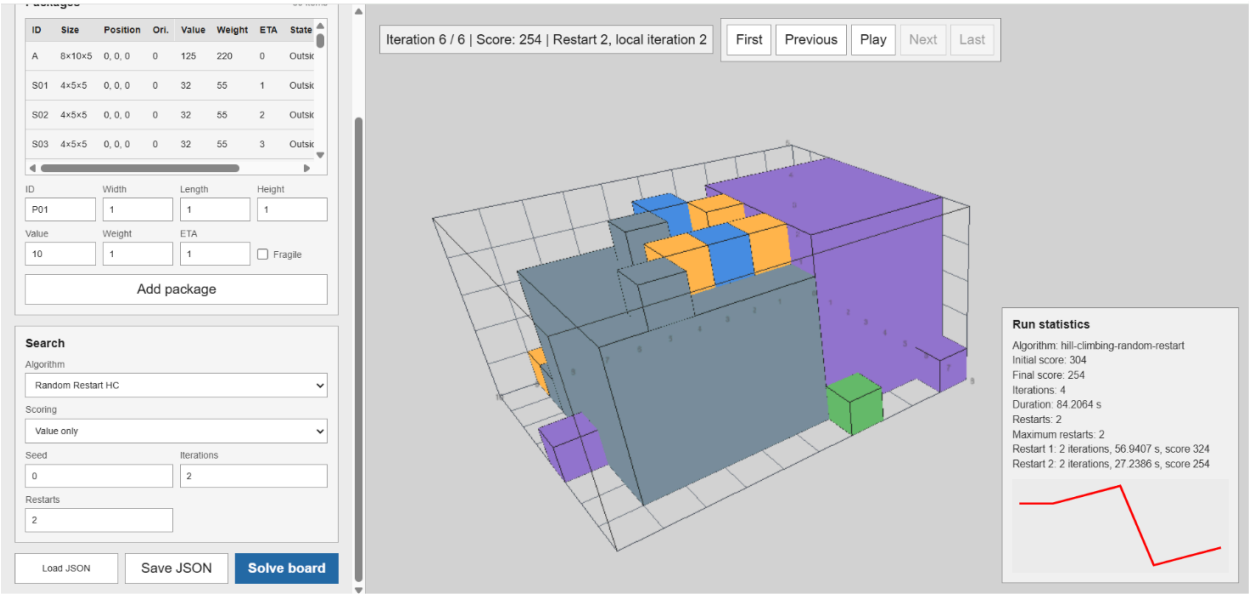
Hasil Pengujian & Analisis:

1. **local_optima**



Berdasarkan hasil pengujian dengan test case local_optimum, algoritma Random Restart Hill Climbing pun masih mengalami stuck pada local optimum.

2. asalklik



Berdasarkan hasil pengujian pada test case asalklik dengan algoritma Random Restart Hill Climbing pada seed 0 terlihat bahwa waktu eksekusi untuk iterasi yang dibatasi sampai 2 iterasi dan 2 restart juga membutuhkan waktu yang sangat lama. Penyebab lama eksekusi ini juga sama dengan pada kasus steepest ascent hill climbing

6. Stochastic Hill Climbing

Stochastic Hill Climbing meng-generate satu random successor dengan cara melakukan randomisasi pada orientasi package atau posisi package atau keberadaan package di truk (menggunakan fungsi gen_random_suc). Lalu mengubah current state menjadi successor tersebut apabila score nya lebih besar.

Sesuai dengan yang dijelaskan di kelas, Stochastic Hill Climbing hanya akan berhenti ketika mencapai maximum iteration, sehingga jumlah iterasi hingga mendapat pencarian berhenti hanya bergantung pada maximum iteration.

Algoritma ini diimplementasikan sebagai berikut:

```
def solve_stochastic(board: dict, max_iter: int = 100) -> dict:
    curr = create_random_individual(board,_inst)
    init = deepcopy(curr)
    history = []

    for iteration in range(0, max_iter):
        candidate = gen_random_suc(curr)

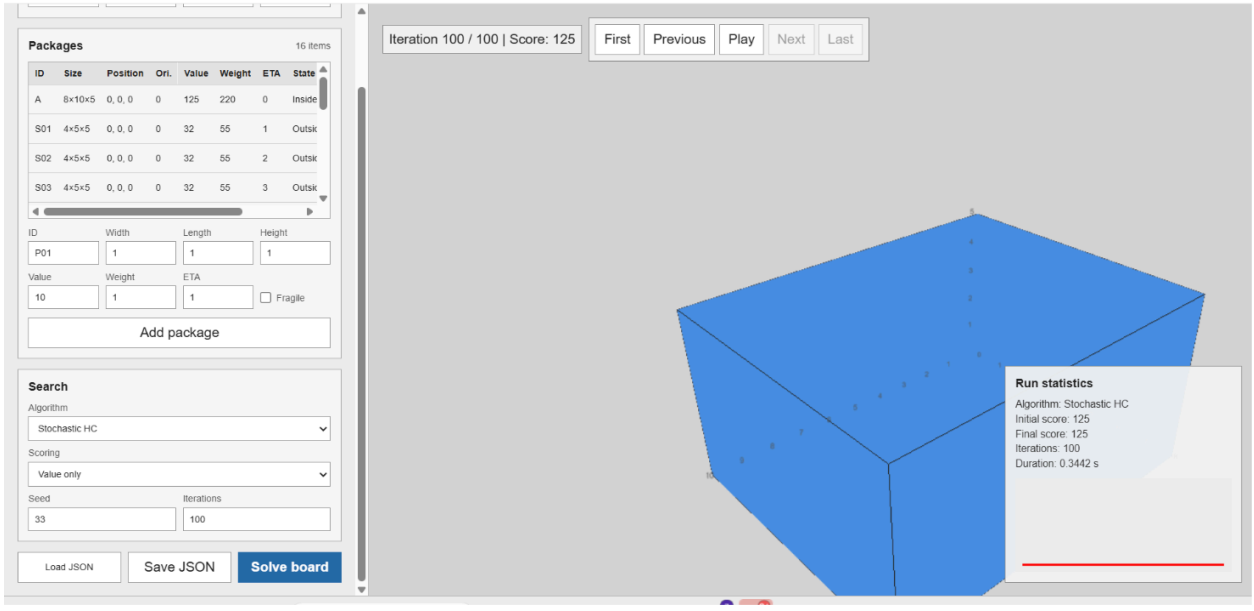
        if scoring(candidate) > scoring(curr):
              curr = candidate

        history.append({
              "iteration" : iteration + 1,
              "score" : scoring(curr),
              "state" : deepcopy(curr)
        })

    return {
        "init" : init,
        "steps" : history,
        "final" : curr,
        "initScore" : scoring(init),
        "finalScore" : scoring(curr),
        "iterations" : len(history)
    }
```

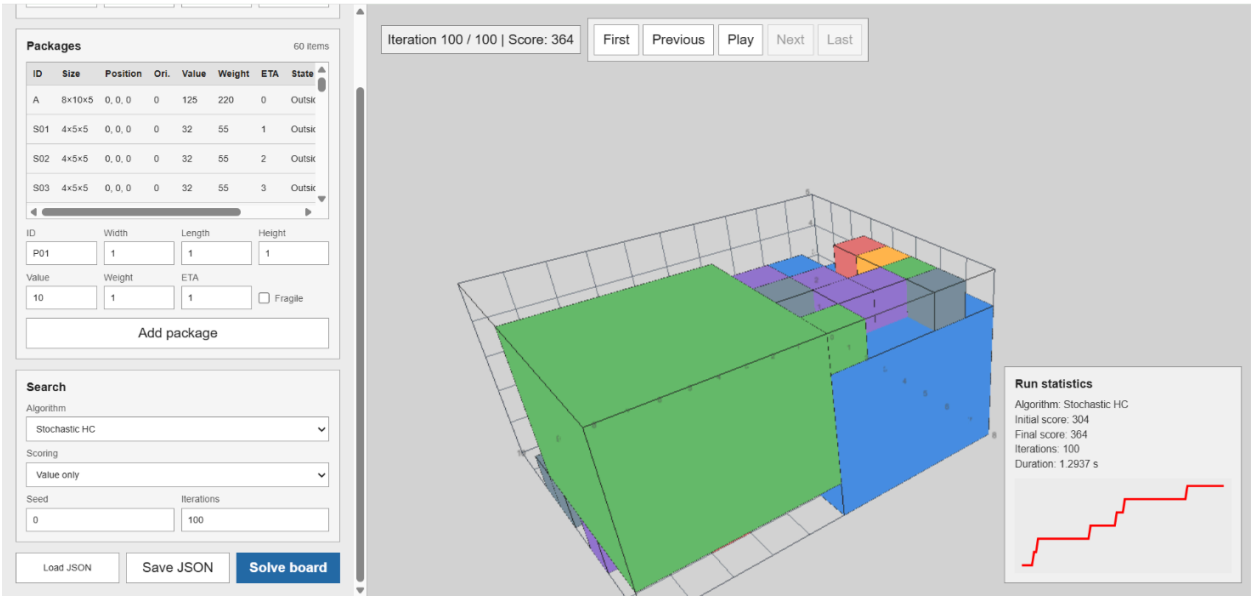
Hasil Pengujian & analisis:

1. local_optima



Berdasarkan hasil pengujian, algoritma Stochastic Hill Climbing sama dengan steepest hill climbing mengalami stuck pada local optimum dengan score 125 selayaknya variasi hill climbing lainnya karena tidak melakukan random walk.

2. asalklik



Berdasarkan hasil pengujian, algoritma Stochastic Hill Climbing dapat melakukan eksekusi 100 iterasi dengan waktu eksekusi yang sangat rendah yaitu hanya 1 detik, jauh lebih cepat dibanding variasi hill climbing lainnya yang butuh waktu mendekati 1 menit. Hal ini karena pembangkitan 1 successor random bisa dilakukan dengan sangat cepat.

7. Simulated Annealing

Simulated annealing meng-generate satu random successor dengan merandomisasi suatu aksi seperti pada stochastic hill climbing dan mengubah current menjadi neighbor apabila E positif atau jika $e^{\frac{\Delta E}{T}}$ melebihi suatu nilai random. Implementasi Algoritma ini dilakukan dengan fungsi sebagai berikut:

```
def solve(board: dict, temperature: float = 100, cooling_rate: float = 0.95, max_iter: int
= 1000 ) -> dict:
    started_at = perf_counter()
    curr = create_random_individual(board,_inst)
    init = deepcopy(curr)
    history = []

    for iteration in range(0, max_iter):
        temperature*=cooling_rate
        successor=gen_random_suc(curr)
        delta_E=scoring(successor)-scoring(curr)

        if temperature <= 0:
             if delta_E > 0:
             exponential_value = None
             else:
             exponential_value = 1.0 if delta_E == 0 else 0.0
        else:
             exponent = delta_E / temperature
             try:
             exponential_value = exp(exponent)
             except OverflowError:
             exponential_value = None

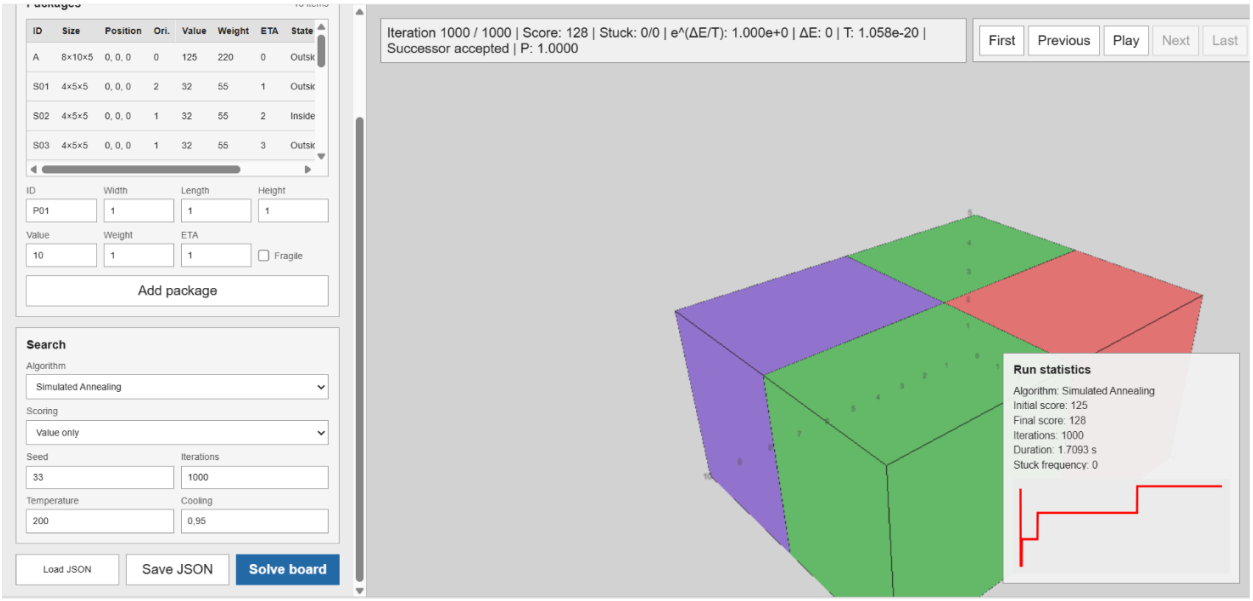
        acceptance_probability = 1.0 if delta_E >= 0 else (exponential_value or 0.0)
        accepted = delta_E >= 0 or random() < acceptance_probability
        if accepted:
             curr = successor

        history.append({
             "iteration" : iteration + 1,
             "score" : scoring(curr),
             "deltaScore": delta_E,
             "temperature": temperature,
            "expDeltaOverTemperature": exponential_value,
            "acceptanceProbability": acceptance_probability,
             "accepted": accepted,
             "state" : deepcopy(curr)
        })

    return {
        "init" : init,
        "steps" : history,
        "final" : curr,
        "initScore" : scoring(init),
        "finalScore" : scoring(curr),
        "iterations" : len(history),
        "duration": perf_counter() - started_at,
    }
```

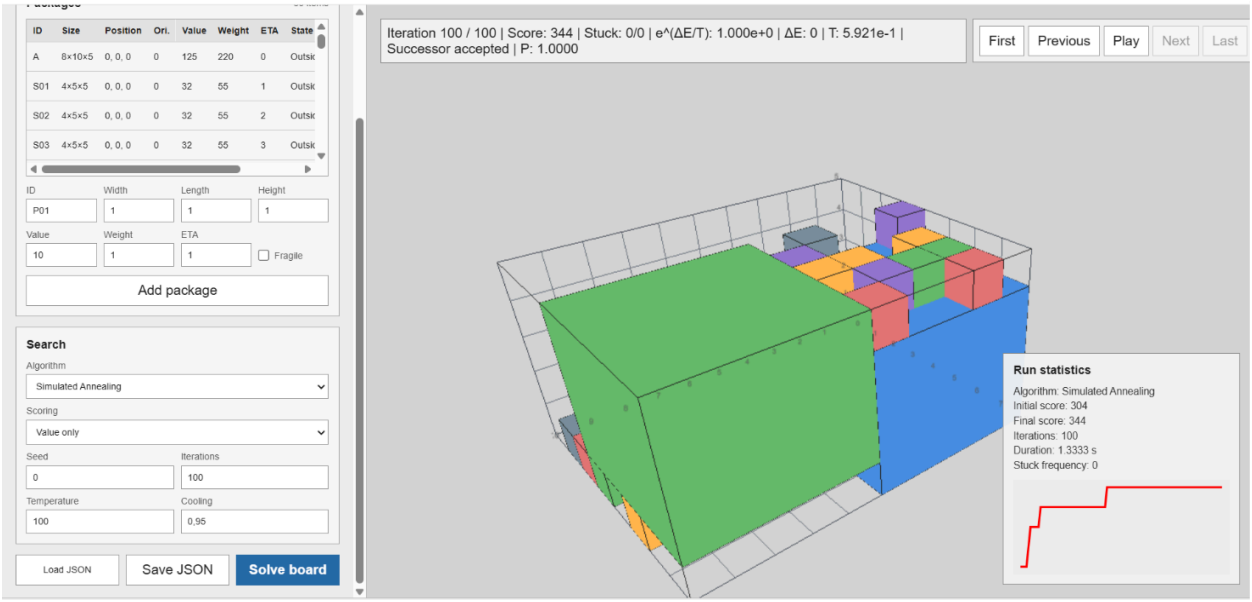
Hasil Pengujian & analisis:

1. local_optima



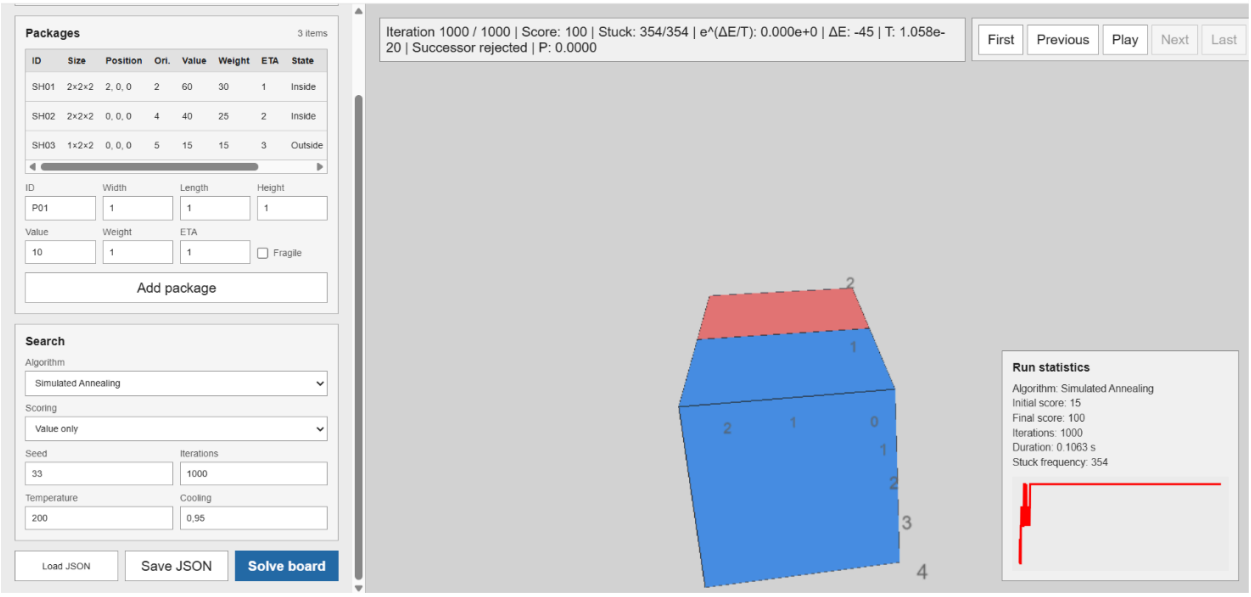
Berdasarkan hasil pengujian, algoritma Simulated Annealing dengan 1000 iterasi berhasil menemukan konfigurasi paket yang valid dengan nilai fungsi objektif sebesar 128 yaitu global optimum. Hasil ini dikarenakan simulated annealing melakukan random walk.

2. asalklik



Berdasarkan hasil pengujian, algoritma Simulated Annealing dapat melakukan eksekusi 100 iterasi dengan waktu eksekusi yang sangat rendah yaitu hanya 1 detik sama seperti stochastic hill climbing.

3. shoulder



Sebenarnya testcase ini tidak relevan pada algoritma simulated annealing, tapi author menampilkannya hanya untuk menunjukkan bahwa nilai stuck berhasil dicatat.

8. Genetic Algorithm

Algoritma Genetika (GA) yang digunakan merepresentasikan setiap individu sebagai sebuah state truk yang memuat status paket, koordinat posisi, dan orientasi. Populasi awal dibangkitkan melalui create_random_individual() urutan paket diacak, lalu setiap paket memiliki peluang 50% untuk dicoba dimasukkan dengan orientasi dan posisi acak yang divalidasi terhadap aturan penempatan.

Kualitas individu dinilai menggunakan fungsi scoring(), yang menghitung nilai paket yang berhasil dimuat sesuai mode penilaian yang dipilih. Pemilihan induk menggunakan Roulette Wheel Selection, individu dengan fitness lebih tinggi memiliki peluang lebih besar untuk terpilih, tetapi pemilihannya tetap bersifat acak.

Dua induk menghasilkan anak melalui onepoint crossover. Anak dibangun dari state kosong dengan mewarisi penempatan paket dari induk pertama sebelum titik potong dan induk kedua setelahnya. Setiap penempatan diperiksa validitasnya, jika penempatan warisan gagal, program mencoba mencari penempatan acak yang valid. Mutasi memiliki peluang 15% pada setiap anak, dengan operasi berupa memindahkan paket, menukar posisi paket, atau mengubah orientasi.

Proses berlangsung sampai jumlah iterasi yang ditentukan tercapai. Skor maksimum dan rata-rata populasi dicatat pada setiap iterasi, lalu solusi yang dikembalikan adalah individu terbaik pada iterasi terakhir.

Pembahasan source code

a. Find_random_valid_placement

   Fungsi ini menempatkan satu paket ke dalam truk dengan mencari kombinasi orientasi dan koordinat. Fungsi menerima state, indeks paket yang akan ditempatkan, dan rng angka acak yang digunakan supaya prosesnya dapat dikontrol oleh seed.

   Pertama, fungsi mengambil dimensi truk dan membuat daftar enam orientasi yang mungkin. Urutan orientasi tersebut diacak. Untuk setiap orientasi, fungsi menghitung dimensi paket setelah diputar menggunakan curr_dimension. Dari hasil itu, fungsi menentukan koordinat maksimum yang masih memungkinkan paket berada di dalam batas truk. Jika paket terlalu besar untuk orientasi tersebut, orientasi itu dilewati.

   Untuk setiap orientasi yang muat, fungsi membuat daftar seluruh koordinat bilangan bulat yang mungkin, lalu mengacak urutannya. Setiap koordinat dicoba pada salinan state menggunakan place_at. Fungsi memakai salinan karena place_at mengubah state saat mencoba menempatkan paket. Jika percobaan dinyatakan valid, state hasil percobaan disalin kembali ke state, lalu fungsi mengembalikan True. Jika semua kombinasi gagal, fungsi mengembalikan False dan state awal tidak diperbarui.

```
def find_random_valid_placement(state: dict, package_index: int, rng: Random) ->
bool:
    truck_dimensions = state["truck"]["dimensions"]
```

```
    orientations = list(range(6))
    rng.shuffle(orientations)

    for orientation in orientations:
        package = deepcopy(state["packages"][package_index])
        package["orientation"] = orientation
        dimensions = curr_dimension(package)
        max_x = truck_dimensions["w"] - dimensions["w"]
        max_y = truck_dimensions["h"] - dimensions["h"]
        max_z = truck_dimensions["l"] - dimensions["l"]

        if max_x < 0 or max_y < 0 or max_z < 0:
            continue

        positions = [
            {"x": x, "y": y, "z": z}
            for x in range(max_x + 1)
            for y in range(max_y + 1)
            for z in range(max_z + 1)
        ]
        rng.shuffle(positions)

        for position in positions:
            candidate = deepcopy(state)
            if place_at(candidate, package_index, orientation, position):
                state.clear()
                state.update(candidate)
                return True

    return False
```

b. Create_random_individual

Fungsi ini membuat satu individu awal berdasarkan papan atau konfigurasi paket yang diberikan. Ia memanggil empty_state untuk menyalin papan dan menandai seluruh paket sebagai belum berada di truk.

Selanjutnya, fungsi mengacak urutan indeks paket. Untuk setiap paket, paket hanya dicoba untuk dimuat jika undiannya memenuhi peluang yang ditentukan di 50%. Paket yang terpilih untuk dicoba kemudian diberikan kepada find_random_valid_placement. Paket yang tidak terpilih, atau yang tidak mendapatkan penempatan valid, tetap memiliki status di luar truk.

```
def create_random_individual(board: dict, rng: Random) -> dict:
    individual = empty_state(board)
    package_indices = list(range(len(individual["packages"])))
    rng.shuffle(package_indices)

    for package_index in package_indices:
        if rng.random() < 0.50:
            find_random_valid_placement(individual, package_index, rng)
```

```
    return individual
```

c. Fitness

Meneruskan penghitungan skor ke scoring(state) di common.py.

```
def fitness(state: dict) -> float:
    return scoring(state)
```

d. Selection

Memilih satu individu sebagai induk dari populasi dengan metode Roulette Wheel Selection. Fungsi menghitung total fitness seluruh individu, lalu mengambil angka acak dari rentang nol sampai total tersebut. Setelah itu, fungsi menelusuri populasi sambil menjumlahkan fitness.

Individu pertama yang jumlah kumulatifnya mencapai atau melewati angka acak tersebut terpilih. Secara probabilistik, individu dengan fitness lebih tinggi memiliki bagian, sehingga peluang terpilihnya juga lebih besar. Contohnya, jika satu individu memiliki fitness 40 dari total fitness 100, peluangnya secara ideal sekitar 40%.

```
def selection(population: list[dict], rng: Random) -> dict:
    total_fitness = 0

    for individual in population:
        total_fitness += fitness(individual)

    if total_fitness <= 0:
        return deepcopy(rng.choice(population))

    random_number = rng.uniform(0, total_fitness)
    cumulative_fitness = 0

    for individual in population:
        cumulative_fitness += fitness(individual)

        if random_number <= cumulative_fitness:
            return deepcopy(individual)

    return deepcopy(population[-1])
```

e. Crossover

Fungsi crossover(parent_a, parent_b, rng) menghasilkan satu anak dengan metode onepoint crossover. Titik potong dipilih di antara indeks paket. Paket sebelum titik potong diwarisi dari parent_a, sedangkan paket pada atau setelah titik potong diwarisi dari parent_b.

Anak dibangun mulai dari empty_state(parent_a), jadi seluruh paket awalnya dianggap berada di luar truk. Untuk setiap indeks paket, fungsi melihat induk yang menjadi sumber pewarisan. Jika paket itu memang berada di dalam truk pada induknya, fungsi mencoba meletakkannya pada anak dengan orientasi dan posisi yang diwarisi. Percobaan dilakukan pada salinan anak dan diperiksa oleh place_at, jika valid, salinan itu menjadi anak sementara yang baru.

Posisi dari dua induk bisa saling bertabrakan ketika digabungkan. Misal paket tidak bisa ditempatkan di posisi warisannya, fungsi mencoba find_random_valid_placement sebagai upaya penempatan alternatif. Jika gagal, paket itu tidak dimasukkan ke anak.

```
def crossover(parent_a: dict, parent_b: dict, rng: Random) -> dict:
    child = empty_state(parent_a)
    package_count = len(child["packages"])

    if package_count < 2:
        crossover_point = package_count
    else:
        crossover_point = rng.randint(1, package_count - 1)

    for package_index in range(package_count):
        if package_index < crossover_point:
            parent = parent_a
        else:
            parent = parent_b

        package = parent["packages"][package_index]

        if not package.get("inTruck", False):
            continue

        candidate = deepcopy(child)
        if place_at(
            candidate,
            package_index,
            package.get("orientation", 0),
            deepcopy(package["position"]),
        ):
            child = candidate
        else:
            repair_candidate = deepcopy(child)
            if find_random_valid_placement(repair_candidate, package_index, rng):
```

```
                child = repair_candidate

    return child
```

f. Mutate

Mutasi memiliki peluang 15% untuk setiap anak. Jika terjadi, GA memilih satu paket lalu menerapkan salah satu operasi secara acak move mencoba hingga 30 posisi baru dengan orientasi, swap menukar posisi dan status pemuatan dua paket atau rotate mencoba orientasi tetangga secara acak sampai menemukan orientasi yang valid.

```
def mutate(individual: dict, rng: Random, mutation_rate: float = 0.15) -> dict:
    mutated = deepcopy(individual)

    if rng.random() >= mutation_rate or not mutated["packages"]:
        return mutated

    operation = rng.choice(["move", "swap", "rotate"])
    package_count = len(mutated["packages"])

    if operation == "move":
        package_index = rng.randrange(package_count)
        current_package = mutated["packages"][package_index]
        trial = deepcopy(mutated)
        trial["packages"][package_index]["inTruck"] = False
        package = trial["packages"][package_index]
        dimensions = curr_dimension(package)
        truck_dimensions = trial["truck"]["dimensions"]
        max_x = truck_dimensions["w"] - dimensions["w"]
        max_y = truck_dimensions["h"] - dimensions["h"]
        max_z = truck_dimensions["l"] - dimensions["l"]

        if max_x >= 0 and max_y >= 0 and max_z >= 0:
            for _ in range(30):
                position = {
                    "x": rng.randrange(max_x + 1),
                    "y": rng.randrange(max_y + 1),
                    "z": rng.randrange(max_z + 1),
                }

                if current_package.get("inTruck", False):
                    if position == current_package.get("position"):
                        continue

                candidate = deepcopy(trial)
                is_placed = place_at(
                    candidate,
                    package_index,
                    package.get("orientation", 0),
                    position,
                )

                if is_placed:
                    return candidate
```

```
    elif operation == "swap" and package_count >= 2:
        first, second = rng.sample(range(package_count), 2)
        trial = deepcopy(mutated)
        trial["packages"][first]["position"], trial["packages"][second]["position"]
= (
            trial["packages"][second]["position"],
            trial["packages"][first]["position"],
        )
        trial["packages"][first]["inTruck"], trial["packages"][second]["inTruck"] =
(
            trial["packages"][second].get("inTruck", False),
            trial["packages"][first].get("inTruck", False),
        )
        if isvalid(trial):
            return trial

    elif operation == "rotate":
        package_index = rng.randrange(package_count)
        current_orientation = mutated["packages"][package_index]["orientation"]
        orientations = list(ROTATION_NEIGHBORS[current_orientation])
        rng.shuffle(orientations)

        for orientation in orientations:
            trial = deepcopy(mutated)
            trial["packages"][package_index]["orientation"] = orientation
            if isvalid(trial):
                return trial

    return mutated
```

g. Next_generation

Membentuk populasi iterasi baru dengan mengulang proses sampai jumlah anak sama dengan jumlah individu pada populasi lama. Pada setiap pengulangan, fungsi memanggil selection dua kali untuk memilih dua induk, lalu memanggil crossover untuk menghasilkan anak. Anak tersebut kemudian diberikan kepada mutate dan hasilnya dimasukkan ke populasi baru.

```
def next_generation(population: list[dict], rng: Random) -> list[dict]:
    next_population = []

    while len(next_population) < len(population):
        parent_a = selection(population, rng)
        parent_b = selection(population, rng)
        child = crossover(parent_a, parent_b, rng)
        next_population.append(mutate(child, rng))

    return next_population
```

h. solve

solve(board, population_size, generations, seed) mengatur jalannya GA dari awal sampai hasil dikembalikan. Fungsi memeriksa parameter dasar, membuat Random(seed), lalu membangun populasi awal dengan memanggil create_random_individual sebanyak population_size. Seed mengatur sumber keacakan.

Setelah populasi awal dibuat, fungsi mencari individu dengan fitness tertinggi dan menyimpannya sebagai init. Fungsi menghitung statistik iterasi ke0, yaitu fitness maksimum dan rata-rata populasi awal. Kemudian, untuk setiap iterasi, fungsi memanggil next_generation, menghitung ulang fitness populasi yang baru, menyimpan skor terbaik beserta statenya ke steps, dan mencatat fitness maksimum serta rata-rata ke generationStats.

Setelah jumlah iterasi selesai, final diambil sebagai individu terbaik di populasi terakhir. Final bisa memiliki fitness lebih rendah daripada init, atau lebih rendah daripada solusi terbaik yang sempat ditemukan pada iterasi sebelumnya karena GA tidak menyimpan hasil terbaik.

```
def solve(
    board: dict,
    population_size: int = 30,
    generations: int = 50,
    seed: int | None = None,
) -> dict:
    if population_size < 1:
        raise ValueError("population_size harus minimal 1")
    if generations < 0:
        raise ValueError("generations tidak boleh negatif")

    rng = Random(seed)
    started_at = perf_counter()
    population = [create_random_individual(board, rng) for _ in range(population_size)]
    initial_scores = [fitness(individual) for individual in population]
    initial_best = deepcopy(max(population, key=fitness))
    history: list[dict] = []
    generation_stats = [
        {
            "generation": 0,
            "maxScore": fitness(initial_best),
            "averageScore": sum(initial_scores) / len(initial_scores),
        }
    ]

    for generation in range(1, generations + 1):
        population = next_generation(population, rng)
        scores = [fitness(individual) for individual in population]
        best_index = max(range(len(population)), key=lambda index: scores[index])
        best_state = deepcopy(population[best_index])
        best_score = scores[best_index]

        history.append(
            {
                "iteration": generation,
```

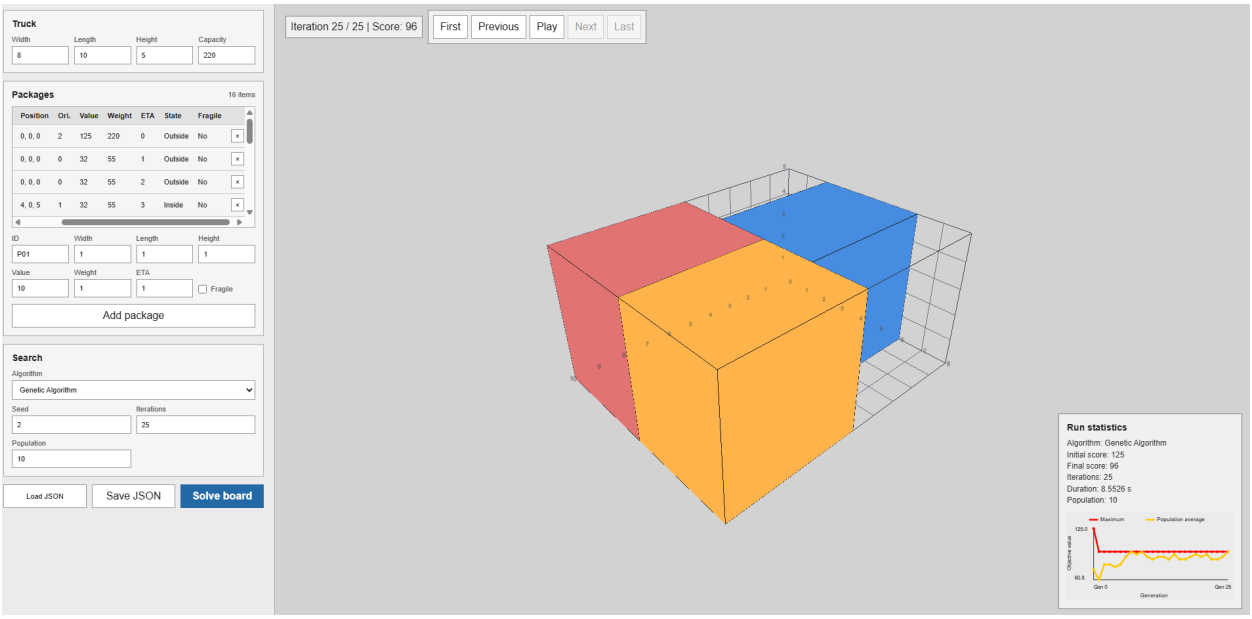
```
                "score": best_score,
                "state": best_state,
            }
        )
        generation_stats.append(
            {
                "generation": generation,
                "maxScore": best_score,
                "averageScore": sum(scores) / len(scores),
            }
        )

    final_best = deepcopy(max(population, key=fitness))

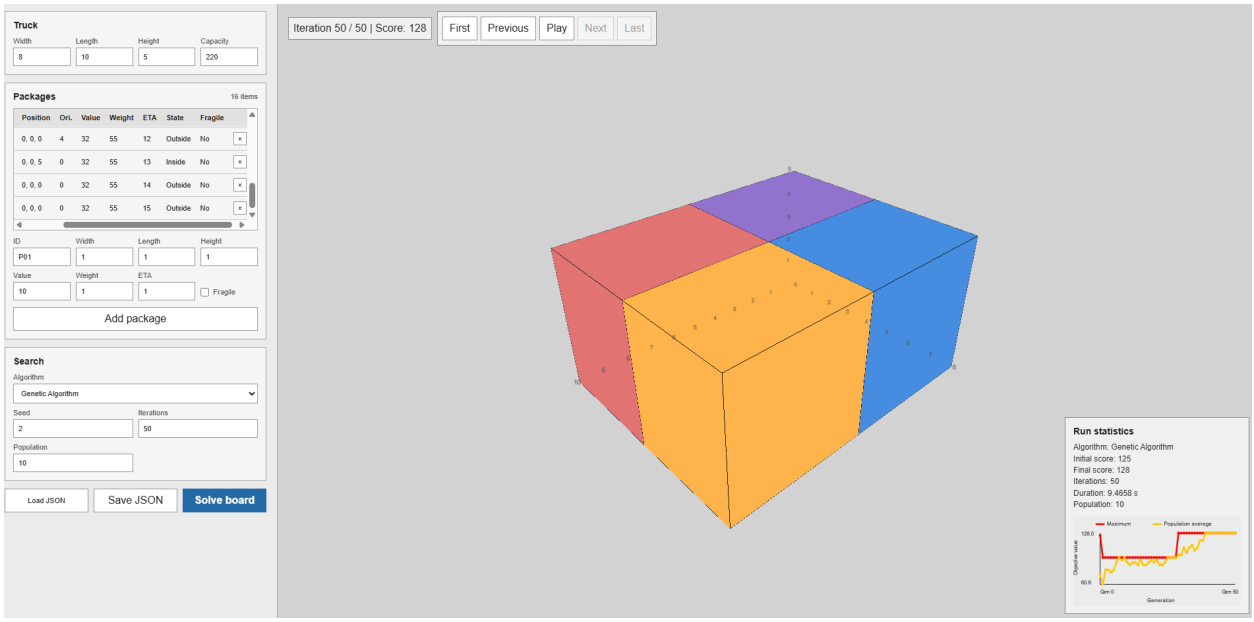
    return {
        "init": initial_best,
        "steps": history,
        "final": final_best,
        "initScore": fitness(initial_best),
        "finalScore": fitness(final_best),
        "iterations": generations,
        "populationSize": population_size,
        "generationStats": generation_stats,
        "durationSeconds": perf_counter() - started_at,
        "seed": seed,
    }
```

Testing: Test case Local_Optimum

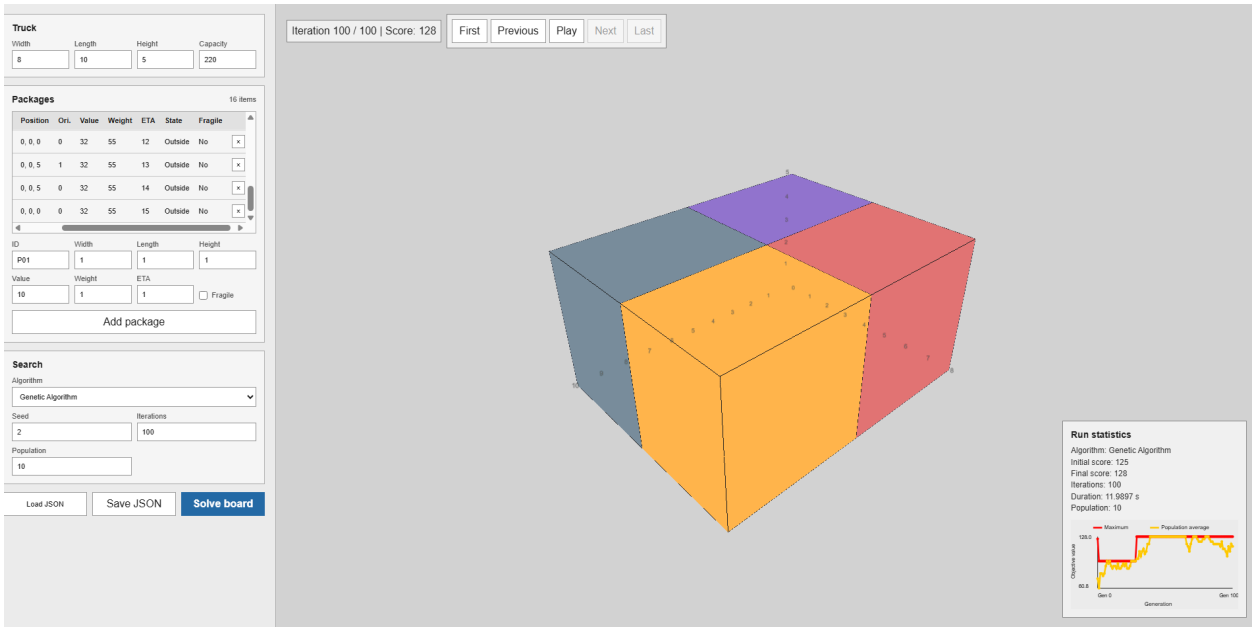
a. Run 1: Populasi 10, iterasi 25, seed 2



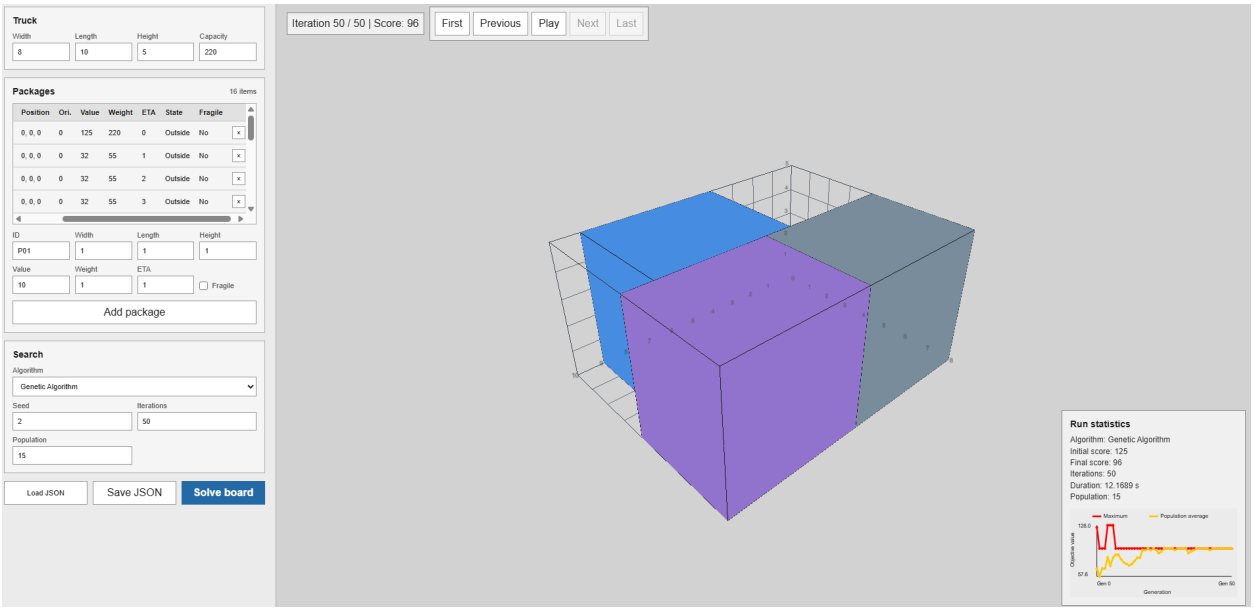
b. Run 2: Populasi 10, iterasi 50, seed 2



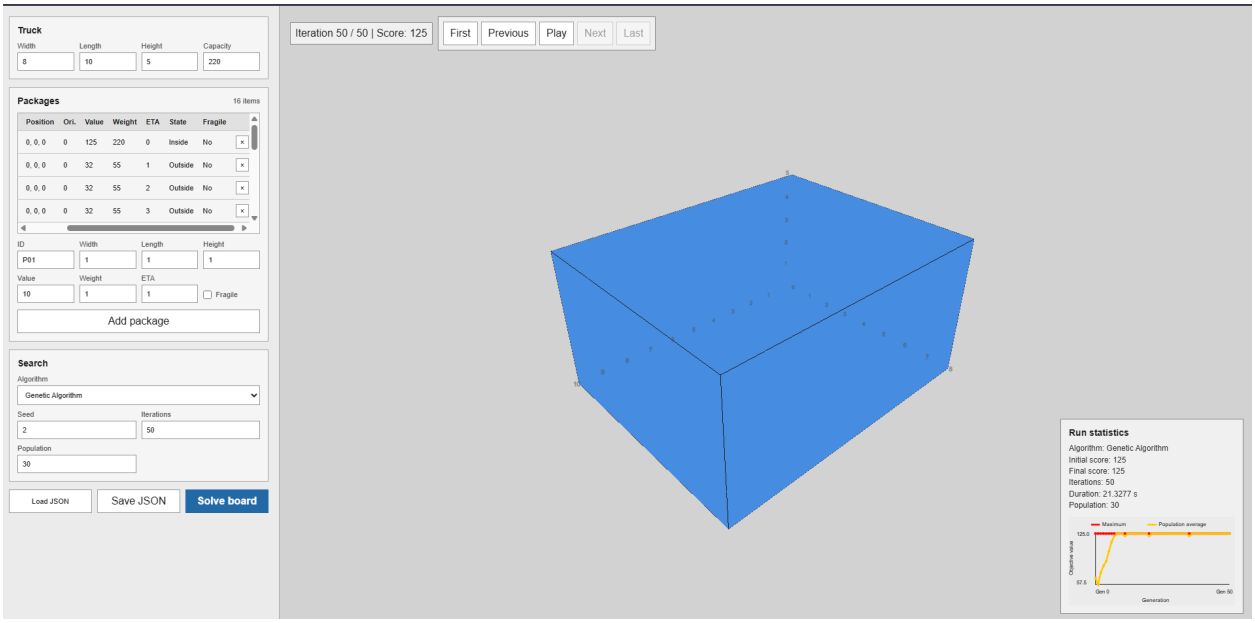
c. Run 3: Populasi 10, iterasi 100, seed 2



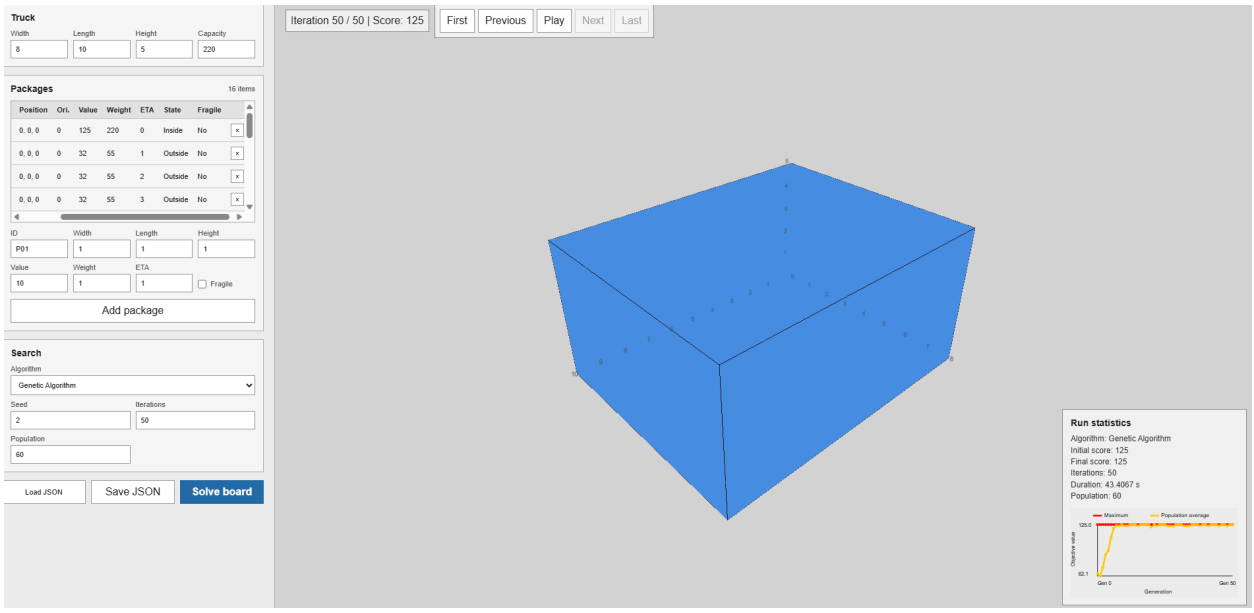
d. Run 4: Populasi 15, iterasi 50, seed 2



e. Run 5: Populasi 30, iterasi 50, seed 2



f. Run 6: Populasi 60, iterasi 50, seed 2



Pembahasan Hasil:

A. Pengaruh jumlah iterasi terhadap skor akhir

Dengan populasi 10, skor naik dari 96 pada 25 iterasi menjadi 128 pada 50 iterasi. iterasi tambahan memberi GA lebih banyak kesempatan untuk menyeleksi induk dan menghasilkan anak melalui crossover serta mutasi.

B. Pengaruh populasi terhadap skor akhir

Pada 50 iterasi, populasi 15 memperoleh skor 96, populasi 30 memperoleh 125, dan populasi 60 juga memperoleh 125. Populasi lebih besar menyediakan lebih banyak kandidat pada tiap iterasi, tetapi tidak menjamin skor akhir lebih tinggi.

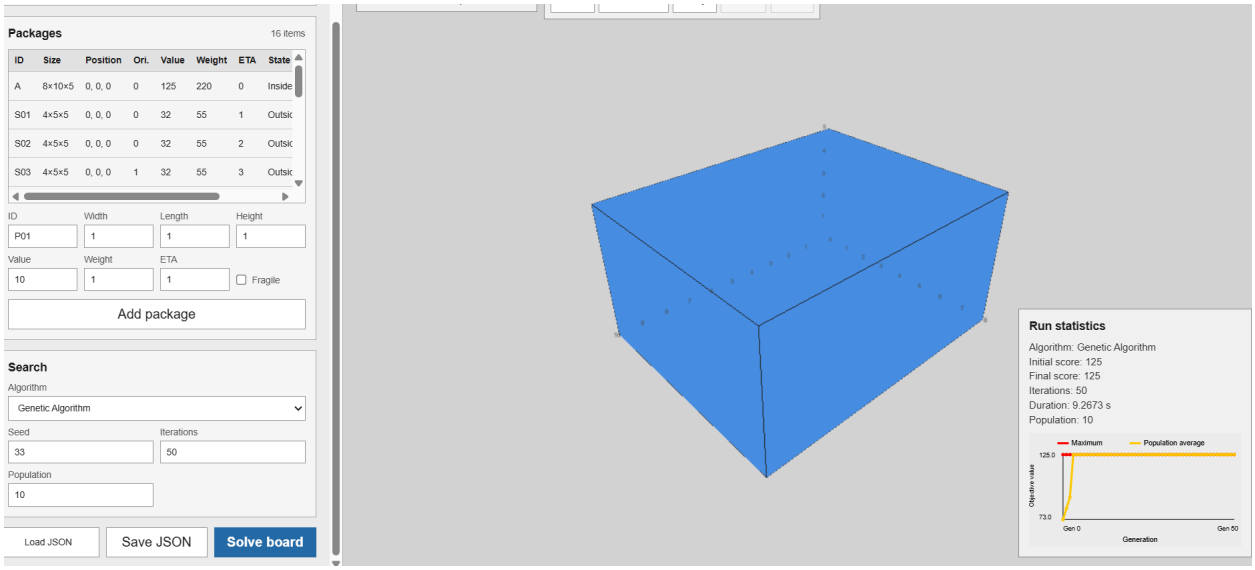
C. Pengaruh seed terhadap skor akhir

Menggunakan

Populasi 10, iterasi 50, seed 33

Di dapat initial score dan final score bernilai 125 yang berarti terjebak dalam local optima, berbeda dengan test run 2 dan 3 yang mendapat global optima dengan menggunakan seed 2.

Hal ini dikarenakan perbedaan seed memengaruhi individu yang dibangkitkan, pemilihan induk, titik crossover dan operasi pada mutasi sehingga jalur pencarian berbeda dan akan memengaruhi hasil akhir meskipun jumlah populasi dan iterasi sama.



D. Konfigurasi terbaik untuk global optimum

Skor tertinggi 128 di dapat di run ke 2 dan ke 3

E. Durasi menaik

Karena semakin banyak iterasi berarti semakin banyak terjadinya siklus pembentukan dan penilaian kepada populasi, dan karena banyak individu yang perlu diseleksi, di crossover dan di mutasi berpengaruh terhadap durasi

9. Perbandingan semua algoritma menggunakan test case Optimum local dengan seed 33

Algoritma	Initial Score	Final Score	Durasi	Jumlah iterasi
Steepest Ascent Hill Climbing	125	125	0,2156	0
Sideway Hill Climbing	125	125	4,3350	50
Random Restart Hill Climbing	125	64	2,5011	10 (dengan 10 restart)

Stochastic Hill Climbing	125	125	0,3442	100
Simulated Annealing	125	128	1,7983	1000
Genetic Algorithm	125	125	8,2673	50

Berdasarkan pengujian seluruh algoritma dengan test case local optimum dengan seed 33, sebagian besar algoritma seperti Ascent Hill Climbing, Sideway Hill Climbing, Stochastic Hill Climbing, serta Genetic Algorithm (GA) tercatat mendapat skor yang stuck di skor akhir sama dengan skor awal yaitu 125 yang berarti gagal mencapai global optimum yang seharusnya bernilai 128, Performa yang kurang optimal juga ditunjukkan oleh algoritma Random Restart Hill Climb dengan mendapat final score 64 dan initial score 125, Sebaliknya hanya Simulated Annealing (SA) yang mendapat global optimum dengan final state 128 dikarenakan oleh mekanisme random walk dengan suhu, sehingga sukses untuk mencapai final skor 128 (global optimum). Selain berhasil mendapatkan global optimum, stochastic juga menjadi algoritma dengan durasi eksekusi paling singkat per iterasinya dengan perbandingan durasi per iterasinya adalah $\frac{1,7983}{1000}$ alias 0,0017983.

A. Kesimpulan dan Saran

1. Kesimpulan

Dalam Tugas Besar 1 IF3170 Inteligensi Artifisial ini, kami telah berhasil mengimplementasikan berbagai macam bentuk local search untuk menyelesaikan permasalahan yang dihadapi special week, yaitu masalah optimisasi posisi pemuatan paket dalam truck-nya. Semua algoritma berhasil membuat state yang merepresentasikan kondisi truck yang valid dan memiliki value yang optimal. Namun, tidak semua algoritma berhasil membuat konfigurasi yang global optimum, karena bisa mengalami stuck di local optimum. Setelah dilakukan pengujian pun didapati bahwa simulated Annealing adalah algoritma yang berhasil dalam menemukan global optimum dengan waktu paling cepat relatif terhadap jumlah iterasi ketika algoritma lain mengalami stuck.

2. Saran

Pengembangan selanjutnya dapat berfokus pada peningkatan efisiensi pembangkitan successor dan strategi pada eksplorasi ruang pencarian. Pembangkitan seluruh successor membutuhkan waktu yang lama sehingga dapat lebih dioptimalkan, pada Genetic Algorithm operator mutation dan crossover dapat dikembangkan lagi sehingga algoritma tidak terjebak di local optimum. Selain itu, pengujian dapat dilakukan dengan lebih banyak variasi seed, ukuran populasi, jumlah iterasi, kompleksitas test case untuk memperoleh perbandingan performa yang lebih representatif dan menentukan konfigurasi parameter yang sesuai bagi setiap algoritma yang diujikan.

Pembagian Tugas

Anggota	Kontribusi
Raysha Erviandika Putra	Algoritma Hill Climbing, Arsitektur backend-frontend.
Zeki Amani	Algoritma Simulated Annealing, Stochastic hill climbing, random restart hill climbing.
Muhammad Daffa Arrizki Yanma	Genetic Algorithm

Referensi

[1] Institut Teknologi Bandung, *IF3170 Inteligensi Artifisial: Beyond Classical Search*, Semester I TA 2026/2027, Institut Teknologi Bandung, 2026.

[2] GeeksforGeeks, "What is Simulated Annealing?," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/. [Accessed: Oct. 20, 2026].

[3] DataCamp, "Genetic Algorithm in Python," *DataCamp*. [Online]. Available: https://www.datacamp.com/tutorial/genetic-algorithm-python. [Accessed: Oct. 20, 2026].

Lampiran

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM :

- Zeki Amani/13524082
- Raysha Erviandika Putra/13524050
- Muhammad Daffa Arrizki Yanma/13524133

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar AI

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

AI/kecerdasan buatan yang digunakan adalah:

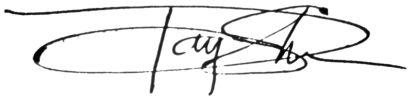
Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	iya	Diskusi alur kerja algoritma GA serta debugging GA
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan: Pembuatan Test case dan Visualisasi	**IYA**	Membuat test Case untuk uji coba & mempercepat pembuatan visualisasi

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Muhammad Daffa Arrizki Yanma
13524133



Raysha Erviandika Putra
13524050



Zeki Amani
13524082